Pontifícia Universidade Católica de Minas Gerais
Curso de Engenharia de Software

Conquista — Sistema de Moeda Estudantil

Lab03S01: Modelagem (versão atualizada após a Sprint 2)

Disciplina: Projeto de Software
Professora: Milena Menezes Adão

Alunos: Felipe Costa Lisboa, Arthur Martins Candido, Gustavo Leonardi Ribeiro e Sofia Fernandes Ferreira Silva

Entregáveis: Diagrama de Casos de Uso, Histórias do Usuário, Diagrama de Classes, Diagrama de Componentes, evolução da arquitetura e matriz de rastreabilidade

Laboratório 3 — Segundo Semestre/2026 · Documento revisado em 26/10/2026

Sobre esta versão

Este documento atualiza o *Lab03S01-modelagem.pdf* (Sprint 1) para refletir a arquitetura efetivamente implementada na Sprint 2, preservando os requisitos da professora, os identificadores originais de casos de uso (UC) e histórias (US) e a modelagem conceitual do sistema completo. O documento original não foi alterado.

Base de conferência. O alinhamento entre modelo e código foi verificado por leitura do código-fonte do repositório (commit *5d253b4*, ramo *master*): modelos SQLAlchemy, schemas Pydantic, routers, services, repositórios, migração Alembic e páginas React. A execução das suítes de teste não faz parte desta revisão.

Convenção de situação usada em todos os diagramas e tabelas:

Situação	Significado
Implementado	Existe no código da Sprint 2 e atende aos critérios da história/caso de uso.
Implementado*	Funcionalidade pronta, mas sem controle de acesso por perfil, pois a autenticação (US01) ainda não existe.
Parcial	Parte dos critérios foi atendida; o restante depende da Sprint 3 (em geral, login e senha).
Pendente	Previsto no enunciado e na modelagem; ainda não implementado.

Autenticação, JWT, APScheduler, e-mail, vantagens, transações e resgates continuam **planejados** e não são apresentados como entregues em nenhuma parte deste documento.

1. Atores

Ator	Descrição
Aluno	Se cadastra, recebe moedas, consulta extrato e troca moedas por vantagens
Professor	Pré-cadastrado pela instituição; envia moedas aos alunos e consulta extrato
Empresa Parceira	Se cadastra, cadastra vantagens e confere cupons de resgate
Sistema (agendamento e notificações)	Ator secundário: credita 1.000 moedas por semestre aos professores e dispara e-mails

O ator secundário, chamado “Sistema (Scheduler/E-mail)” na Sprint 1, passa a ser identificado como “Sistema de agendamento e notificações”, sem mudança de responsabilidade.

2. Diagrama de Casos de Uso

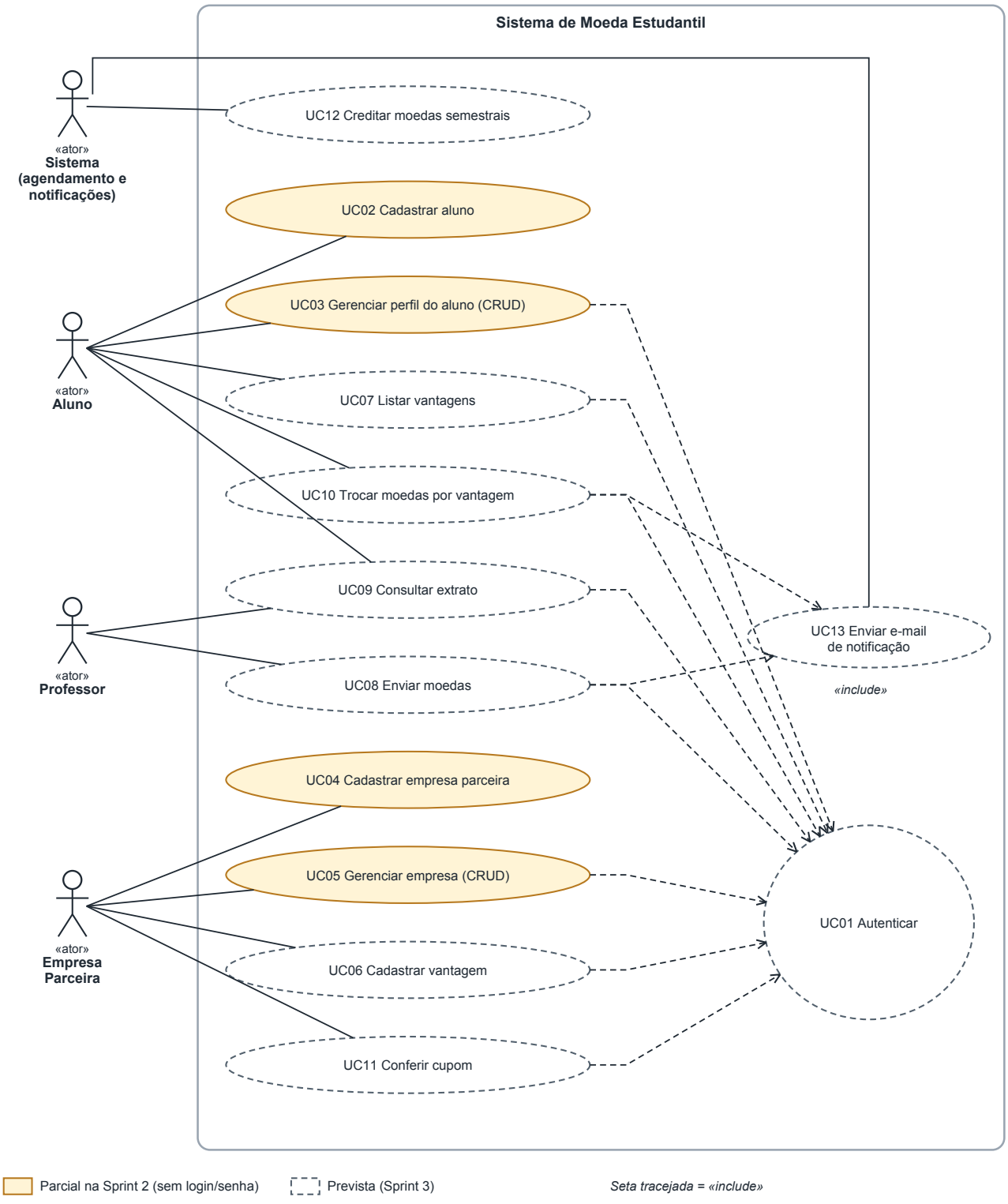


Figura 1 — Diagrama de Casos de Uso (situação na Sprint 2)

2.1 Descrição resumida dos casos de uso

ID	Caso de uso	Ator	Pré-condição	Fluxo principal	Pós-condição	Situação
UC01	Autenticar	Aluno, Professor, Empresa	Possuir login e senha	Informa login/senha → sistema valida → cria sessão conforme perfil	Usuário autenticado	Pendente
UC02	Cadastrar aluno	Aluno	-	Informa nome, e-mail, CPF, RG, endereço, curso, escolhe instituição (pré-cadastrada), define login/senha	Aluno criado com saldo 0	Parcial (sem login/senha)
UC03	Gerenciar perfil do aluno (CRUD)	Aluno	Autenticado	Visualiza, edita e exclui os próprios dados (CPF não editável; exclusão com confirmação)	Cadastro atualizado ou removido	Implementado*
UC04	Cadastrar empresa parceira	Empresa	-	Informa dados da empresa e define login/senha	Empresa criada	Parcial (sem login/senha)
UC05	Gerenciar empresa (CRUD)	Empresa	Autenticada	Visualiza, edita e exclui os próprios dados (CNPJ não editável; exclusão com confirmação)	Cadastro atualizado ou removido	Implementado*
UC06	Cadastrar vantagem	Empresa	Autenticada	Informa nome, descrição, foto e custo em moedas	Vantagem disponível para resgate	Pendente
UC07	Listar vantagens	Aluno	Autenticado	Visualiza vantagens com foto, descrição, custo e empresa	-	Pendente
UC08	Enviar moedas	Professor	Autenticado; saldo suficiente	Seleciona aluno, informa valor e motivo (obrigatório) → debita professor, credita aluno, registra transação	Aluno notificado por e-mail	Pendente
UC09	Consultar extrato	Aluno, Professor	Autenticado	Exibe saldo atual e histórico de transações (professor: envios; aluno: recebimentos e trocas)	-	Pendente
UC10	Trocar moedas	Aluno	Autenticado; saldo ≥ custo	Seleciona vantagem → debita saldo → gera código → registra resgate	E-mail de cupom ao aluno e e-mail de conferência à empresa, ambos com o código	Pendente
UC11	Conferir cupom	Empresa	Autenticada	Informa o código apresentado pelo aluno → sistema valida e marca como utilizado	Resgate marcado como utilizado	Pendente
UC12	Creditar moedas semestrais	Sistema	Início do semestre	Soma 1.000 moedas ao saldo corrente de cada professor (acumulativo)	Saldos atualizados	Pendente
UC13	Enviar e-mail de notificação	Sistema	Incluído por UC08 e UC10	Envia ao aluno o aviso de moedas recebidas (professor, quantidade e motivo) e, no resgate, o cupom ao aluno e a conferência à empresa, ambos com o código	E-mails enviados	Pendente

Fluxos alternativos principais: saldo insuficiente (UC08/UC10) → operação recusada com mensagem; motivo vazio (UC08) → bloqueia envio; credenciais inválidas (UC01) → acesso negado; CPF/e-mail já cadastrado (UC02) → erro de validação; código de cupom inválido ou já usado (UC11) → recusado.

Observações da revisão. (a) As linhas de UC03, UC05 e UC13 já constavam no diagrama e na matriz de rastreabilidade da Sprint 1, mas não na tabela de descrições; foram descritas aqui a partir dos requisitos e das histórias US03, US13, US04 e US07. (b) UC07 recebeu a relação «include» com UC01, pois a Sprint 1 já exigia aluno autenticado como pré-condição. (c) Nenhum caso de uso foi removido ou acrescentado.

3. Histórias do Usuário

Prioridade: **A** = alta (núcleo / sprints 2–3), **M** = média, **B** = baixa. O texto e os critérios de aceite abaixo são os da Sprint 1, sem alteração.

Autenticação

US01 - Login (A)

Como aluno, professor ou empresa parceira, quero fazer login com usuário e senha para acessar as funções do meu perfil.

Critérios de aceite: Credenciais válidas redirecionam para a área do perfil correspondente. Credenciais inválidas exibem erro sem informar qual campo está errado. Senhas são armazenadas com hash. Rotas protegidas exigem autenticação e respeitam o perfil.

Aluno

US02 - Cadastro de aluno (A)

Como aluno, quero me cadastrar informando nome, e-mail, CPF, RG, endereço, instituição e curso para participar do sistema de mérito.

Critérios de aceite: instituição escolhida em lista pré-cadastrada; CPF e e-mail únicos; todos os campos obrigatórios validados; saldo inicial 0.

US03 - Editar/excluir meu cadastro (A)

Como aluno, quero visualizar, editar e excluir meus dados para mantê-los atualizados.

Critérios de aceite: edição não permite alterar CPF; exclusão exige confirmação.

US04 - Notificação de moedas recebidas (A)

Como aluno, quero ser notificado por e-mail ao receber moedas para saber quem me reconheceu e por quê.

Critérios de aceite: e-mail contém professor, quantidade e motivo.

US05 - Extrato do aluno (A)

Como aluno, quero consultar meu saldo e histórico (recebimentos e trocas) para acompanhar minhas moedas.

Critérios de aceite: exibe saldo atual; transações ordenadas da mais recente para a mais antiga, com data, tipo, valor e descrição.

US06 - Listar vantagens (A)

Como aluno, quero ver as vantagens disponíveis, com foto, descrição e custo, para escolher o que trocar.

Critérios de aceite: mostra empresa, custo em moedas e indica se meu saldo é suficiente.

US07 - Trocar moedas por vantagem (A)

Como aluno, quero trocar minhas moedas por uma vantagem para usufruir do benefício na empresa parceira.

Critérios de aceite: bloqueia se saldo < custo; debita o valor; gera código único; envia e-mail de cupom ao aluno e e-mail de conferência à empresa, ambos com o código; operação atômica (transação de banco).

Professor

US08 - Enviar moedas (A)

Como professor, quero enviar moedas a um aluno da minha instituição, com um motivo, para reconhecer bom desempenho.

Critérios de aceite: exige saldo suficiente; motivo obrigatório; aluno deve pertencer à mesma instituição; debita professor e credita aluno na mesma transação.

US09 - Extrato do professor (A)

Como professor, quero consultar meu saldo e os envios realizados para controlar minha distribuição.

Critérios de aceite: mostra saldo e lista de envios com aluno, valor, motivo e data.

US10 - Crédito semestral (M)

Como professor, quero receber 1.000 moedas a cada semestre, acumulando o saldo não utilizado, para continuar reconhecendo meus alunos.

Critérios de aceite: crédito executado uma vez por semestre; soma ao saldo corrente; registrado no extrato.

US11 - Professor pré-cadastrado (M)

Como instituição parceira, quero enviar a lista de professores (nome, CPF, departamento) para que eles já existam no sistema vinculados à instituição.

Critérios de aceite: professor sempre vinculado a uma instituição; carga inicial via seed/importação.

Empresa parceira

US12 - Cadastro de empresa parceira (A)

Como empresa, quero me cadastrar com login e senha para oferecer vantagens aos alunos.

Critérios de aceite: dados obrigatórios validados; CNPJ/e-mail únicos.

US13 - Editar/excluir empresa (A)

Como empresa, quero editar e excluir meus dados para mantê-los corretos.

US14 - Cadastrar vantagem (A)

Como empresa, quero cadastrar vantagens com descrição, foto e custo em moedas para atrair alunos.

Critérios de aceite: nome, descrição, foto e custo > 0 obrigatórios; vantagem vinculada à empresa logada; empresa pode editar/desativar suas vantagens.

US15 - Conferir cupom (M)

Como empresa, quero validar o código apresentado pelo aluno para confirmar a troca presencial.

Critérios de aceite: código válido e não utilizado é marcado como usado; código inválido ou repetido é recusado.

Dados de apoio

US16 - Instituições pré-cadastradas (M)

Como aluno, quero selecionar minha instituição em uma lista para me vincular corretamente.

Critérios de aceite: lista carregada do banco (seed inicial).

3.1 Acompanhamento da implementação (Sprint 2)

Classificação feita critério a critério. Como a autenticação ainda não existe, nenhuma história que exige login e senha foi marcada como totalmente concluída.

História	Funcionalidade	Situação	Justificativa
US01	Login	Pendente	Sem autenticação; *senha_hash* existe no modelo, mas é opcional e nunca preenchida.
US02	Cadastro de aluno	Parcial	Atendidos: dados, endereço, instituição em lista, CPF/e-mail únicos, validação, saldo inicial 0. Pendente: definição de login/senha.
US03	Edição e exclusão de aluno	Implementado*	Atendidos: CPF imutável e exclusão com confirmação. Sem restrição ao “meu” cadastro (depende de US01).
US04	Notificação por e-mail	Pendente	Não há serviço de e-mail.
US05	Extrato do aluno	Pendente	Não há transações.
US06	Listar vantagens	Pendente	Entidade Vantagem ainda não existe.
US07	Trocar moedas por vantagem	Pendente	Sem vantagens, resgates ou cupons.
US08	Enviar moedas	Pendente	Sem professores nem transações.
US09	Extrato do professor	Pendente	Sem professores nem transações.
US10	Crédito semestral	Pendente	APScheduler não adotado.
US11	Professor pré-cadastrado	Pendente	O seed atual carrega apenas instituições de ensino.
US12	Cadastro de empresa	Parcial	Atendidos: dados obrigatórios validados, CNPJ/e-mail únicos. Pendente: login e senha.
US13	Edição e exclusão de empresa	Implementado*	CRUD completo com confirmação e CNPJ imutável; sem restrição à empresa logada (depende de US01).
US14	Cadastrar vantagem	Pendente	Vantagem não implementada.
US15	Conferir cupom	Pendente	Cupom não implementado.
US16	Instituições pré-cadastradas	Implementado	Endpoint */api/instituicoes* lista instituições carregadas do banco pelo seed.

* Funcionalidade pronta, sem controle de acesso por perfil até a Sprint 3.

4. Diagrama de Classes

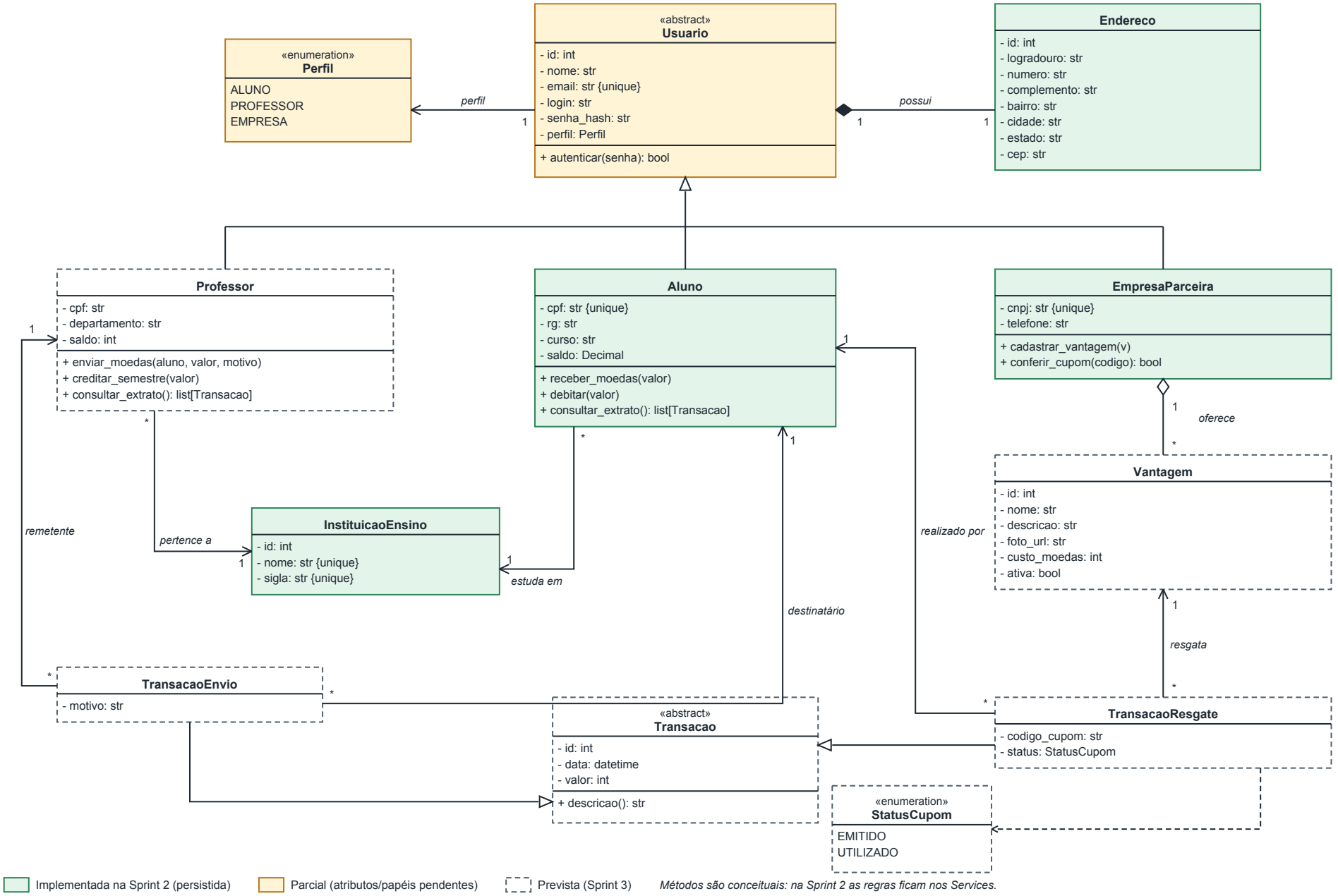


Figura 2 — Diagrama de Classes (modelo conceitual completo; situação na Sprint 2 indicada por cor e borda)

Decisões de modelagem

• `Usuario` centraliza e-mail, senha e perfil (herança), atendendo ao requisito de autenticação para os três tipos de ator. Na Sprint 2 a tabela `usuarios` já existe, com `tipo` restrito a ALUNO e EMPRESA e `senha_hash` opcional.

• `Transacao` é abstrata com duas especializações: `TransacaoEnvio` (professor → aluno, com motivo obrigatório) e `TransacaoResgate` (aluno → vantagem, com código de cupom). O extrato é a consulta dessas transações por usuário.

• O vínculo professor–instituição é obrigatório, conforme o enunciado; o vínculo aluno–instituição é obrigatório e já é imposto pelo banco (chave estrangeira não nula com `ON DELETE RESTRICT`).

• O envio só é permitido se `professor.saldo >= valor` e `valor > 0`; resgate só se `aluno.saldo >= vantagem.custo_moedas`.

• **Saldo protegido.** `saldo` é atributo privado. Os schemas de cadastro e edição do aluno não aceitam saldo; o cadastro cria o saldo com 0 e a alteração ficará restrita às operações de moeda da Sprint 3.

• **Unicidade.** E-mail único em `Usuario`, CPF único em `Aluno` e CNPJ único em `EmpresaParceira`, garantidos por restrição `UNIQUE` no banco e por verificação prévia nos services (resposta 409). CPF e CNPJ não são editáveis após o cadastro.

• `Endereco` está ligado a `Usuario` (composição 1–1), e não mais diretamente ao aluno, como na Sprint 1.

• Os métodos do diagrama são conceituais. Na Sprint 2 as classes persistidas são entidades sem comportamento; as regras ficam nos Services.

Modelo conceitual × entidades persistidas na Sprint 2

Classe conceitual	Situação	Persistência na Sprint 2	Diferenças em relação à Sprint 1
Usuario	Parcial	Tabela usuarios: id, nome, email (único), tipo, senha_hash (opcional), criado_em, atualizado_em	nome passou para Usuario (antes em Aluno/Professor); login ainda não existe (e-mail identifica o usuário); tipo aceita ALUNO e EMPRESA
Aluno	Implementada	Tabela alunos (PK = usuario_id): cpf, rg, curso, instituicao_id, saldo Numeric(14,2)	saldo conceitual é inteiro; no banco é decimal com 2 casas — a definir na Sprint 3
EmpresaParceira	Implementada	Tabela empresas_parceiras (PK = usuario_id): cnpj, telefone	nome_fantasia corresponde a Usuario.nome; telefone foi acrescentado
InstituicaoEnsino	Implementada	Tabela instituicoes_ensino: id, nome (único), sigla (única)	sigla acrescentada
Endereco	Implementada	Tabela enderecos: usuario_id (único), logradouro, numero, complemento, bairro, cidade, estado, cep	associado a Usuario
Perfil	Parcial	Restrição CHECK em usuarios.tipo	PROFESSOR ainda não aceito
Professor, Vantagem, Transacao, TransacaoEnvio, TransacaoResgate, StatusCupom	Prevista	Não persistidas	Sem alteração conceitual

5. Diagrama de Componentes (arquitetura MVC)

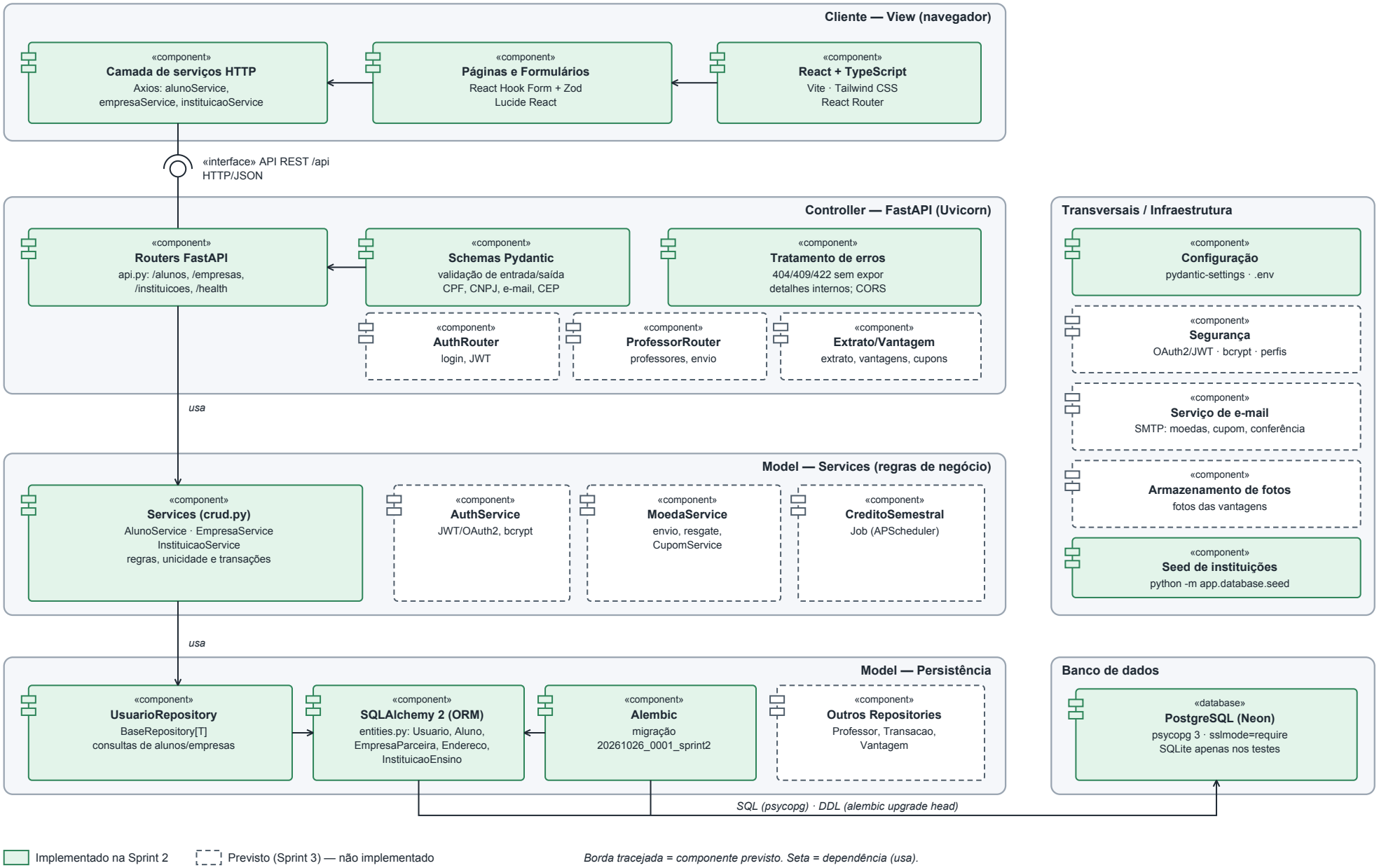


Figura 3 — Diagrama de Componentes (componentes implementados em verde; previstos com borda tracejada)

6. Descrição da arquitetura

Stack implementada na Sprint 2. Front-end: React, TypeScript, Vite, Tailwind CSS, Axios, React Router, React Hook Form, Zod e Lucide React. Back-end: Python, FastAPI, Pydantic, SQLAlchemy 2, Alembic e Uvicorn. Persistência: PostgreSQL hospedado no Neon, com padrão Repository sobre SQLAlchemy e migrações Alembic.

Fluxo de uma requisição: React → Axios (HTTP/JSON, interface API REST em /api) → Routers FastAPI → Services → Repositories → SQLAlchemy → PostgreSQL. A aplicação é organizada em camadas, com front-end e back-end independentes, de modo compatível com o MVC.

Execução: `uvicorn app.main:app --reload --port 8000` (API) e `npm run dev` (cliente, porta 5173).

Componente	Responsabilidade	Situação
View — React + TypeScript	Telas de cadastro, listagem, busca, consulta e edição de alunos e empresas; dashboard; navegação (React Router); estilo (Tailwind CSS)	Implementado
Formulários — React Hook Form + Zod	Validação no cliente de campos obrigatórios, CPF, CNPJ e e-mail	Implementado
Serviços HTTP — Axios	alunoService, empresaService e instituicaoService; consomem a API REST	Implementado
Controllers — Routers FastAPI	Entrada HTTP em `/api/alunos`, `/api/empresas`, `/api/instituicoes`; injeção da sessão de banco; códigos de resposta	Implementado
Schemas — Pydantic	Validação e serialização de entrada e saída (CPF, CNPJ, e-mail, CEP, tamanhos)	Implementado
Services	AlunoService, EmpresaService e InstituicaoService: regras de negócio, unicidade e controle de transação (rollback e erro 409/404)	Implementado
Repository	UsuarioRepository (sobre BaseRepository): consultas de alunos, empresas, instituições e unicidade de e-mail	Implementado
Model — SQLAlchemy	Entidades Usuario, Aluno, EmpresaParceira, Endereco e InstituicaoEnsino, com restrições e chaves estrangeiras	Implementado
Alembic	Migração `20261026_0001_sprint2` cria o schema; `python -m app.database.seed` carrega as instituições	Implementado
PostgreSQL (Neon)	Banco da aplicação; SQLite isolado apenas nos testes automatizados	Implementado
Segurança (OAuth2/JWT, bcrypt, perfis)	Autenticação e controle de acesso por perfil (US01)	Previsto
AuthRouter, ProfessorRouter, Extrato/Vantagem	Novos controllers para login, professores, extrato, vantagens e cupons	Previsto
MoedaService, CupomService, AuthService	Envio e resgate de moedas, geração de cupom, autenticação	Previsto
CreditoSemestralJob (APScheduler)	Crédito semestral de 1.000 moedas (UC12)	Previsto
Serviço de e-mail (SMTP)	Notificações de moedas, cupom e conferência (UC13)	Previsto
Armazenamento de fotos	Foto das vantagens (UC06)	Previsto

Mapeamento MVC → implementação. Model = entidades SQLAlchemy + Services + Repositories; View = cliente React; Controller = Routers FastAPI. Não são utilizados templates Jinja2. Pydantic (servidor) e Zod (cliente) validam as duas fronteiras.

Segurança na Sprint 2. Os CRUDs não têm autenticação; portanto a API não deve ser publicada antes da Sprint 3. Conflitos de integridade geram rollback e resposta 409 sem detalhes internos, e o CORS aceita apenas origens configuradas.

7. Evolução da Arquitetura — Sprint 1 para Sprint 2

1. Arquitetura inicialmente proposta. A Sprint 1 propôs uma aplicação MVC monolítica em Python + FastAPI: View com templates Jinja2, Controllers como routers FastAPI, Services, Repositories (DAO) com SQLAlchemy/Alembic, PostgreSQL, APScheduler, OAuth2/JWT e SMTP.

2. Substituição de Jinja2 por React + TypeScript. A View deixou de ser renderizada no servidor e passou a ser um cliente React com TypeScript, construído com Vite, Tailwind CSS, React Router, React Hook Form, Zod e Lucide React. O papel de View do MVC foi preservado.

3. Separação entre front-end e back-end. Passam a existir dois projetos independentes (`frontend/` e `backend/`), executados e testados separadamente. O back-end não serve mais HTML.

4. Comunicação por API REST. O cliente usa Axios para consumir uma API JSON sob `/api`. Pydantic valida a entrada e a saída no servidor; o CORS libera apenas as origens configuradas.

5. Adoção de SQLAlchemy e Repository. A persistência segue o plano da Sprint 1: SQLAlchemy 2 como ORM e repositórios isolando as consultas. Na Sprint 2 existe um único repositório concreto (UsuarioRepository, sobre BaseRepository), suficiente para os dois CRUDs; os demais virão com as novas entidades.

6. Uso de PostgreSQL no Neon. O banco é um PostgreSQL gerenciado no Neon, acessado com psycopg 3 e `sslmode=require`. Os testes usam SQLite isolado, sem tocar a instância Neon.

7. Migrações com Alembic. O schema é versionado pela migração `20261026_0001_sprint2`. Um seed idempotente carrega as instituições pré-cadastradas (PUC Minas, UFMG, CEFET-MG).

8. Funcionalidades entregues na Sprint 2. Dashboard com indicadores vindos da API; CRUD de alunos e de empresas parceiras com busca, validação de CPF, CNPJ, e-mail e campos obrigatórios; unicidade de CPF, CNPJ e e-mail; CPF e CNPJ imutáveis; saldo inicial 0 protegido; confirmação antes de excluir; seleção de instituições pré-cadastradas; tratamento de carregamento, erros e listas vazias.

9. Componentes previstos para as próximas etapas. Autenticação (OAuth2/JWT, hash de senha, perfis), professores pré-cadastrados, envio de moedas, extrato, vantagens com foto, resgate com cupom, serviço de e-mail (SMTP) e crédito semestral com APScheduler. Nenhum desses itens está implementado.

8. Rastreabilidade (requisito → caso de uso → história → classe → componente)

Requisito do enunciado	Caso de uso	História	Classes principais	Componentes	Situação
Cadastro de aluno	UC02, UC03	US02, US03	Aluno, Usuario, Endereco, InstituicaoEnsino	React (páginas/formulários), Routers, Schemas, AlunoService, UsuarioRepository, SQLAlchemy	UC02/US02 parcial; UC03/US03 implementado*
Instituições pré-cadastradas para seleção	UC02	US16	InstituicaoEnsino	InstituicaoService, UsuarioRepository, Alembic (seed), PostgreSQL	Implementado
Professores pré-cadastrados com instituição	-	US11	Professor, InstituicaoEnsino	ProfessorRouter, Repository de Professor, seed	Pendente
1.000 moedas/semestre acumuláveis	UC12	US10	Professor	CreditoSemestralJob (APScheduler)	Pendente
Envio de moedas com motivo	UC08	US08	TransacaoEnvio, Professor, Aluno	MoedaService, ProfessorRouter	Pendente
E-mail ao aluno ao receber moedas	UC13	US04	TransacaoEnvio	Serviço de e-mail (SMTP)	Pendente
Extrato (aluno e professor)	UC09	US05, US09	Transacao	Extrato/VantagemRouter, Repository de Transacao	Pendente
Cadastro de empresa	UC04, UC05	US12, US13	EmpresaParceira, Usuario, Endereco	React, Routers, Schemas, EmpresaService, UsuarioRepository, SQLAlchemy	UC04/US12 parcial; UC05/US13 implementado*
Cadastro e listagem de vantagens	UC06, UC07	US06, US14	Vantagem, EmpresaParceira	Extrato/VantagemRouter, Armazenamento de fotos	Pendente
Troca de moedas + e-mails com código	UC10, UC13	US07	TransacaoResgate, Vantagem, Aluno	MoedaService, CupomService, Serviço de e-mail	Pendente
Conferência pelo parceiro	UC11	US15	TransacaoResgate	CupomService, Extrato/VantagemRouter	Pendente
Login e senha para todos	UC01	US01	Usuario, Perfil	AuthRouter, AuthService, Segurança (OAuth2/JWT, bcrypt)	Pendente

Os identificadores de requisitos, casos de uso e histórias são os da Sprint 1. Foram acrescentadas duas linhas — instituições pré-cadastradas (US16) e listagem de vantagens (UC07/US06) — que já existiam como UC/US, mas não constavam na matriz original.

9. Conclusão — alinhamento entre modelagem e implementação

A modelagem da Sprint 1 permanece válida como visão conceitual do sistema completo. A revisão alterou a camada de apresentação (Jinja2 → React + TypeScript), tornou explícita a separação entre front-end e back-end e registrou a pilha realmente adotada, mantendo o padrão em camadas e a aderência ao MVC.

Alinhado. Os cinco elementos persistidos (Usuario, Aluno, EmpresaParceira, InstituicaoEnsino e Endereco), seus relacionamentos, a herança de Usuario, a obrigatoriedade da instituição para o aluno, as regras de unicidade de e-mail, CPF e CNPJ e a proteção do saldo correspondem ao código.

Divergências registradas (a decidir na Sprint 3). (1) O login ainda não existe; o e-mail é o único identificador. (2) O saldo é inteiro no modelo conceitual e decimal com 2 casas no banco. (3) O perfil PROFESSOR ainda não é aceito pela restrição de `usuarios.tipo`. (4) Endereço ficou associado a Usuario, e não a Aluno.

Pendente para a Sprint 3. Autenticação e controle de acesso (US01), professores, transações de envio e resgate, vantagens, cupons, e-mails e crédito semestral. Esses itens permanecem na modelagem conceitual, mas não estão implementados.

Ressalva. O alinhamento foi conferido por leitura do código-fonte no commit 5d253b4; deve ser reconferido a cada nova migração.